

INFORME TÉCNICO

Proyecto ProdeUTN

Plataforma de Pronósticos Deportivos

TPI — Programación 4 · UTN FRVM · 2025

1. Introducción y Tecnologías

ProdeUTN es una aplicación web de pronósticos deportivos. Los usuarios se registran, pronostican resultados de partidos de fútbol, acumulan puntos según sus aciertos y compiten en rankings globales o grupos privados.

Arquitectura

API REST monolítica con frontend embebido (Spring Boot serve archivos estáticos).

Stack tecnológico

Tecnología	Descripción y uso en el proyecto
Java 17	Versión LTS. Records, Stream API, Switch expressions, Text blocks.
Spring Boot 3.3.0	Autoconfiguration, Embedded Tomcat, Starter POMs. Framework principal.
Spring Data JPA + Hibernate 6	ORM + repositorios. Reduce el código de persistencia con JpaRepository.
Spring Security 6	SecurityFilterChain, BCryptPasswordEncoder, integración JWT stateless.
PostgreSQL	Base de datos relacional principal.
JWT (JJWT 0.12.5)	Autenticación stateless. Token contiene username, rol y expiración (24h).
Lombok	Reduce boilerplate: @Getter, @Setter, @Builder, @Data, etc.
MapStruct	Mapeo automático Entity ↔ DTO generado en tiempo de compilación.
BCrypt	Hasheo de contraseñas con factor de costo 10, salt automático.
Maven 3.9+	Gestor de dependencias y build del proyecto.

2. Estructura del Backend

El backend contiene 57 archivos fuente Java organizados en capas funcionales:

Paquete	Contenido
entity/	12 archivos — Entidades JPA + enums (modelo de datos)
dto/request/ + dto/response/	16 archivos — DTOs de entrada (con validaciones) y salida
mapper/	5 interfaces — Mapeadores MapStruct Entity ↔ DTO
repository/	8 interfaces — Repositorios Spring Data JPA
service/interfaces/ + service/impl/	14 archivos — Contratos e implementaciones de lógica de negocio
controller/	7 clases — Controladores REST (@RestController)
security/	4 clases — Seguridad JWT
config/	2 clases — SecurityConfig + WebConfig
exception/	1 clase — GlobalExceptionHandler (@RestControllerAdvice)
scheduler/	2 clases — Sincronización con API externa de fútbol

3. POM.XML — Dependencias y Plugins

Define las dependencias del proyecto, la versión de Java (17) y los plugins de compilación.

Dependencias principales

Dependencia	Propósito
spring-boot-starter-web	Tomcat embebido, Spring MVC, Jackson. Habilita los <code>@RestController</code> .
spring-boot-starter-data-jpa	Hibernate, Spring Data JPA, HikariCP (pool de conexiones).
spring-boot-starter-security	Spring Security para proteger endpoints con JWT y filtros.
spring-boot-starter-validation	Hibernate Validator. Activa <code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@Min</code> en DTOs.
postgresql (runtime)	Driver JDBC para PostgreSQL. Solo necesario al ejecutar.
jjwt-api / jjwt-impl / jjwt-jackson	Librería JWT: interfaces, implementación y serialización JSON.
lombok (provided)	Genera getters, setters, builders. No se incluye en el JAR final.
mapstruct	Genera implementaciones de mapeo en tiempo de compilación.
spring-boot-starter-test (test)	JUnit 5, Mockito, AssertJ para testing.

Orden crítico en `annotationProcessorPaths`

1. Lombok primero (genera los getters/setters que MapStruct va a consumir).
2. `lombok-mapstruct-binding` (actúa de conector entre ambos).
3. MapStruct processor (genera implementaciones usando los getters de Lombok).

Si el orden es incorrecto, MapStruct no verá los getters y fallará en compilación.

4. Configuración — application.properties

Centraliza toda la configuración. Usa variables de entorno con valores por defecto mediante la sintaxis `${VAR:default}`.

Datasource (PostgreSQL)

```
spring.datasource.url=jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5433}/${DB_NAME:prode_db}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
```

JPA / Hibernate

```
spring.jpa.hibernate.ddl-auto=update    (crea/modifica tablas automáticamente)
spring.jpa.properties.hibernate.jdbc.time_zone=UTC
```

JWT

```
jwt.secret=${JWT_SECRET:...}    (mínimo 256 bits / 32 caracteres)
jwt.expiration=86400000          (24 horas en milisegundos)
```

Football API

```
football.api.base-url=https://api.football-data.org/v4
football.api.token=${FOOTBALL_API_TOKEN:...}
```

CORS y servidor

```
app.cors.allowed-origins=${CORS_ORIGINS:*}
server.port=8080
```

5. Capa de Entidades — Modelo de Datos

Clases Java con anotaciones JPA que representan tablas en la base de datos. 8 entidades + 4 enums = 12 archivos.

Entidades JPA

Entidad	Tabla BD	Campos y relaciones clave
Usuario	usuarios	UUID, username (UNIQUE), email (UNIQUE), passwordHash, rol (enum), createdAt. Relaciones: List<Pronostico>, List<GrupoPrivado>, List<GrupoMiembro>.
Equipo	equipos	UUID, nombre (UNIQUE), escudoUrl (nullable), pais (nullable). Sin @OneToMany con Partido para evitar N+1 queries.
Fecha	fechas	UUID, nombre (UNIQUE), estado (EstadoFecha), createdAt. Método recalcularEstado() calcula el estado según los partidos hijos.
Partido	partidos	UUID, fecha (ManyToOne), equipoLocal, equipoVisitante, inicioUtc (Instant), estado, goles, tendencia, apiExternalId (UNIQUE). Métodos: estaBloqueado(), calcularTendencia().
Pronostico	pronosticos	UUID, usuario+partido (ManyToOne). golesLocalPred, golesVisitantePred, puntos (0/1/3), createdAt (updatable=false), updatedAt. UNIQUE (usuario_id, partido_id). Método: calcularPuntos().
GrupoPrivado	grupos_privados	UUID, creador (ManyToOne), nombre, codigoInvitacion (8 chars UNIQUE), createdAt.
GrupoMiembro	grupo_miembros	Clave primaria compuesta @EmbeddedId(grupoid + usuarioid). Tabla de relación muchos-a-muchos entre usuarios y grupos.
ApiSyncLog	api_sync_log	UUID, tipo (FIXTURE/RESULTADO), ejecutadoAt, partidosProcesados, status (OK/ERROR), errorMsg. Auditoría de sincronizaciones.

Enumeraciones

Enum	Valores y uso
Rol	USER, ADMIN — guardado como STRING. Controla permisos en SecurityConfig.
Tendencia	LOCAL, EMPATE, VISITANTE — base del motor de puntuación (RF6.3).
EstadoPartido	POR_JUGARSE → EN_JUEGO → FINALIZADO. Controla el ciclo de vida del partido.
EstadoFecha	PROGRAMADA, EN_JUEGO, FINALIZADA. Calculado automáticamente con recalcularEstado().

6. Capa de DTOs — Data Transfer Objects

Separan el modelo interno de la API pública. Los Request tienen validaciones jakarta (@NotBlank, @Email, @Min). Los Response tienen @Builder para construirlos fácilmente.

Request DTOs — Validaciones de entrada

DTO	Campos y validaciones
LoginRequestDTO	@NotBlank username, @NotBlank password.
RegisterRequestDTO	@NotBlank @Size(3,50) username, @NotBlank @Email email, @NotBlank @Size(min=6) password.
EquipoRequestDTO	@NotBlank @Size(max=100) nombre, escudoUrl (opcional), @Size(max=60) pais (opcional).
FechaRequestDTO	@NotBlank @Size(max=100) nombre.
PartidoRequestDTO	@NotNull fechalId, equipoLocalId, equipoVisitanteId (UUIDs) y inicioUtc (Instant UTC).
PronosticoRequestDTO	@NotNull partidoid, @NotNull @Min(0) golesLocalPred, @NotNull @Min(0) golesVisitantePred.
ResultadoRequestDTO	@NotNull @Min(0) golesLocal, @NotNull @Min(0) golesVisitante.
GrupoPrivadoRequestDTO	@NotBlank @Size(max=80) nombre.
UnirseGrupoRequestDTO	@NotBlank codigoInvitacion.

Response DTOs — Salida al cliente

DTO	Campos principales
AuthResponseDTO	token (JWT), username, rol. Devuelto en login y registro.
EquipoResponseDTO	id, nombre, escudoUrl, pais.
FechaResponseDTO	id, nombre, estado (String — no enum).
PartidoResponseDTO	id, fechalId, fechaNombre, equipoLocal/Visitante (anidados), inicioUtc, estado, goles, tendencia, bloqueado (calculado en tiempo real).
PronosticoResponseDTO	id, partido + equipos anidados, golesLocalPred/VisitantePred, puntos, resultado real, username, createdAt, updatedAt.
GrupoPrivadoResponseDTO	id, nombre, codigoInvitacion, creadorUsername, cantidadMiembros.
RankingResponseDTO	posicion, usuarioid, username, totalPuntos, totalPlenos.

7. Capa de Mapeadores — MapStruct

MapStruct genera implementaciones de mapeo Entity ↔ DTO en tiempo de compilación. Más eficiente que reflection (BeanUtils) y con errores detectados al compilar, no al ejecutar.

Mapeadores implementados

Mapper	Características destacadas
EquipoMapper	toResponse(), toEntity() ignorando id. updateFromDto() con NullValuePropertyMappingStrategy.IGNORE permite PATCH parcial (solo actualiza campos no nulos).
FechaMapper	estado mapeado con expression java(fecha.getEstado().name()). toEntity() ignora id, estado, createdAt y partidos.
PartidoMapper	uses = {EquipoMapper.class}. Mapea fechaId, fechaNombre, estado, tendencia (ternario null-safe) y bloqueado = partido.estaBloqueado().
PronosticoMapper	uses = {EquipoMapper.class}. Mapea datos anidados del partido y usuario: equipoLocal/Visitante, estadoPartido, golesLocal/Visitante reales, username.
GrupoPrivadoMapper	creadorUsername = grupo.creador.username, cantidadMiembros = grupo.getMembros().size().

8. Capa de Repositorios — Spring Data JPA

Interfaces que Spring implementa automáticamente. Extienden `JpaRepository<T, UUID>` y proveen CRUD básico más queries por nombre de método y `@Query` JPQL.

Repositorio	Métodos y queries destacadas
UsuarioRepository	<code>findByUsername()</code> , <code>findByEmail()</code> , <code>existsByUsername()</code> , <code>existsByEmail()</code> .
EquipoRepository	<code>findByName()</code> , <code>existsByName()</code> , <code>findByNameContainingIgnoreCase()</code> , <code>@Query tienePartidosAsociados(UUID)</code> .
FechaRepository	<code>findByEstado()</code> , <code>findAllOrderByCreatedAtAsc()</code> , <code>@Query tienePartidos(UUID)</code> .
PartidoRepository	<code>findByFechaIdOrderByInicioUtcAsc()</code> , <code>findByApiExternalId()</code> , <code>existsByApiExternalId()</code> , <code>findByEstadoAndInicioUtcBefore()</code> , <code>@Query tienePronosticos(UUID)</code> .
PronosticoRepository	<code>findByUsuarioIdAndPartidoId()</code> para UPSERT. <code>@Query</code> con JOIN FETCH para mis pronósticos. <code>findByPartidoId()</code> para el motor de puntos. <code>findRankingGlobal()</code> y <code>findRankingByGrupo()</code> : GROUP BY, SUM(CASE WHEN puntos=3), ORDER BY puntos DESC / plenos DESC / primerPronostico ASC.
GrupoPrivadoRepository	<code>findByCodigoInvitacion()</code> , <code>existsByCodigoInvitacion()</code> .
GrupoMiembroRepository	<code>findByGrupoid()</code> , <code>findByUsuarioId()</code> , <code>existsByGrupoidAndUsuarioId()</code> . ID compuesto: <code>GrupoMiembroId (@EmbeddedId)</code> .
ApiSyncLogRepository	<code>findTop10OrderByEjecutadoAtDesc()</code> para los últimos 10 logs de sincronización.

9. Capa de Servicios — Lógica de Negocio

Los servicios implementan las reglas de negocio. Usan `@Service` + `@Transactional` (`readOnly=true` para consultas, sin `readOnly` para escritura). El rollback es automático ante excepciones no chequeadas.

AuthServiceImpl

`registrar()`: valida unicidad de username y email, hashlea password con BCrypt, genera JWT.
`login()`: verifica credenciales con `passwordEncoder.matches()`. Devuelve siempre 'Credenciales inválidas' (mensaje genérico para evitar enumeración de usuarios).

EquipoServiceImpl

CRUD completo. `actualizar()` con update parcial (solo campos no nulos del DTO). `eliminar()` verifica RF2.3: no puede tener partidos asociados.

FechaServiceImpl

CRUD. `actualizar()` y `eliminar()` solo si está en estado PROGRAMADA (RF3.2). Listado ordenado por `createdAt` ascendente.

PartidoServiceImpl

`crear()` valida equipos distintos (RF4.1). `registrarResultado()` es el método más importante: guarda goles, calcula tendencia, cambia a FINALIZADO, ejecuta motor de puntuación sobre todos los pronósticos del partido y recalcula estado de la fecha.

PronosticoServiceImpl

`guardar()` implementa UPSERT: busca pronóstico existente y crea o modifica. Valida estado POR_JUGARSE y bloqueo de 30 min (RF1). `listarDeTerceros()` solo si `partido.estaBloqueado() == true` (RF5.3).

GrupoPrivadoServiceImpl

`crear()` genera código único de 8 chars con bucle de hasta 10 intentos. Agrega al creador como primer miembro automáticamente (RF8.1). `unirse()` verifica código válido y no-duplicado (RF8.3).

RankingServiceImpl

`rankingGlobal()` y `rankingGrupo()` mapean `List<Object[]>` de las queries JPQL. `mapearRanking()` asigna posición 1-indexed. Usa `(Number).longValue()` por compatibilidad de casteo con el driver de PostgreSQL.

10. Capa de Controladores — API REST

7 controladores `@RestController` que reciben requests HTTP y delegan en los servicios. Usan `ResponseEntity` para controlar el código HTTP de respuesta.

Controlador	Base URL	Endpoints
AuthController	<code>/api/auth</code>	POST <code>/register</code> (201), POST <code>/login</code> (200). Sin autenticación (<code>permitAll</code>).
EquipoController	<code>/api/equipos</code>	GET <code>/</code> (lista o busca por <code>?nombre=</code>), GET <code>/ {id}</code> , POST <code>/</code> (ADMIN, 201), PUT <code>/ {id}</code> (ADMIN), DELETE <code>/ {id}</code> (ADMIN, 204).
FechaController	<code>/api/fechas</code>	GET <code>/</code> (lista o filtra por <code>?estado=</code>), GET <code>/ {id}</code> , POST <code>/</code> (ADMIN, 201), PUT <code>/ {id}</code> (ADMIN), DELETE <code>/ {id}</code> (ADMIN, 204).
PartidoController	<code>/api/partidos</code>	GET <code>/</code> (por <code>?fechaId=</code>), GET <code>/ {id}</code> , POST <code>/</code> (ADMIN, 201), PUT <code>/ {id}</code> (ADMIN), DELETE <code>/ {id}</code> (ADMIN, 204), PATCH <code>/ {id}/en-juego</code> (ADMIN), PATCH <code>/ {id}/resultado</code> (ADMIN).
PronosticoController	<code>/api/pronosticos</code>	POST <code>/</code> (UPSERT), GET <code>/mis-pronosticos</code> , GET <code>/mis-pronosticos/filtrar?estado=</code> , GET <code>/recientes-acertados</code> , GET <code>/partido/ {id}</code> . Usuario ID proviene del token JWT.
GrupoPrivadoController	<code>/api/grupos</code>	POST <code>/</code> (crear, 201), POST <code>/unirse</code> , GET <code>/mis-grupos</code> , GET <code>/ {id}</code> .
RankingController	<code>/api/ranking</code>	GET <code>/global</code> , GET <code>/pozo</code> , GET <code>/grupo/ {grupoid}</code> .

11. Capa de Seguridad — JWT

Autenticación stateless. El servidor no guarda sesiones: cada request lleva el token en el header Authorization: Bearer <token>.

Estructura de un JWT

- Header: algoritmo de firma (HS256) y tipo (JWT).
- Payload: subject (username), claim 'rol', issuedAt, expiration.
- Firma: HMAC-SHA256(header.payload, secretKey). Verifica integridad.

Clases del módulo de seguridad

JwtUtil

@Component. Genera tokens con generarToken(username, rol), extrae datos con extraerUsername()/extraerRol(), valida con esValido(). Usa la clave secreta de application.properties (mínimo 256 bits).

JwtFilter

@Component, OncePerRequestFilter. Intercepta cada request: lee header Authorization, extrae y valida el token, crea UsernamePasswordAuthenticationToken y lo registra en SecurityContextHolder.

UserDetailsServiceImpl

@Service. Implementa UserDetailsService. loadUserByUsername() convierte el Usuario JPA en UserDetails de Spring Security.

UsuarioContextService

@Component. Helper que lee el username del SecurityContextHolder y devuelve el Usuario completo desde BD. Evita exponer el ID en las requests.

Reglas de autorización — SecurityConfig

Patrón de URL	Acceso requerido	Descripción
/api/auth/**	permitAll	Registro y login, sin token.
GET /api/equipos, /fechas, /partidos/**	authenticated	Cualquier usuario con token válido.
POST/PUT/DELETE /api/equipos, /fechas, /partidos/**	hasRole('ADMIN')	Solo administrador.
PATCH /api/partidos/**	hasRole('ADMIN')	Iniciar partido y cargar resultado.
/api/pronosticos, /ranking, /grupos/**	authenticated	Cualquier usuario autenticado.
/, /index.html, /css/**, /js/**	permitAll	Archivos estáticos del frontend.

12. Manejo de Excepciones — GlobalExceptionHandler

@RestControllerAdvice que captura todas las excepciones y las convierte en respuestas JSON con timestamp, status y mensaje de error. Formato: { timestamp, status, error }.

Excepción	HTTP Status	Casos de uso
IllegalArgumentException	400 BAD REQUEST	Errores de negocio: nombre duplicado, pronóstico tardío, partido bloqueado, etc.
NoSuchElementException	404 NOT FOUND	Recurso inexistente: equipo, fecha, partido, usuario no encontrado.
MethodArgumentNotValidException	400 BAD REQUEST	Falla de @Valid en DTOs. Devuelve mapa campo → mensaje de todos los campos inválidos.
AccessDeniedException	403 FORBIDDEN	Sin permisos para la operación (usuario normal intentando acción de ADMIN).
Exception (catch-all)	500 INTERNAL ERROR	Cualquier error no controlado. Mensaje genérico para no exponer detalles internos.

13. Scheduler — Sincronización Automática

Integración con la API [football-data.org](https://api.football-data.org/). @EnableScheduling activo en ProdeApplication. Los logs de cada ejecución se persisten en la tabla `api_sync_log`.

FootballApiClient.java

@Component que usa HttpClient de Java 11. Header X-Auth-Token con el token de la API. Parsea respuestas JSON con `ObjectMapper.readTree()`. Métodos: `getFixture(competitionCode)` para partidos programados, `getMatchesFinalizados(competitionCode)` para resultados de los últimos 3 días.

FootballSyncScheduler.java — 3 jobs automáticos

Job	Frecuencia	Función
fixtureSyncJob	Lunes 06:00 UTC	Importa partidos programados de todas las ligas. Idempotente por <code>apiExternalId</code> : si ya existe, lo omite. Crea equipos y fechas si no existen.
autoTransitionJob	Cada 15 minutos	Busca partidos <code>POR_JUGARSE</code> cuya <code>inicioUtc</code> ya pasó y los marca <code>EN_JUEGO</code> . Recalcula estado de la fecha.
enJuegoJob	Cada 15 minutos	Consulta resultados <code>FINISHED</code> de la API. Guarda resultado, calcula tendencia, ejecuta motor de puntuación, marca <code>FINALIZADO</code> . Persiste <code>ApiSyncLog</code> con status OK o ERROR.

14. Endpoints Completos de la API — 31 endpoints

Módulo	Método	Endpoint	Acceso	RF
AUTH	POST	/api/auth/register	Público	-
AUTH	POST	/api/auth/login	Público	-
EQUIPOS	GET	/api/equipos	Autenticado	2.1
	GET	/api/equipos/{id}	Autenticado	2.1
	POST	/api/equipos	ADMIN	2.2
	PUT	/api/equipos/{id}	ADMIN	2.2
	DELETE	/api/equipos/{id}	ADMIN	2.3
FECHAS	GET	/api/fechas	Autenticado	3.1
	GET	/api/fechas/{id}	Autenticado	3.1
	POST	/api/fechas	ADMIN	3.1
	PUT	/api/fechas/{id}	ADMIN	3.2
	DELETE	/api/fechas/{id}	ADMIN	3.2
PARTIDOS	GET	/api/partidos?fechaId=	Autenticado	4.4
	GET	/api/partidos/{id}	Autenticado	4.4
	POST	/api/partidos	ADMIN	4.1
	PUT	/api/partidos/{id}	ADMIN	4.2
	DELETE	/api/partidos/{id}	ADMIN	4.5
	PATCH	/api/partidos/{id}/en-juego	ADMIN	4.3
	PATCH	/api/partidos/{id}/resultado	ADMIN	6.1
PRONÓSTICOS	POST	/api/pronosticos	Autenticado	5.1
	GET	/api/pronosticos/mis-pronosticos	Autenticado	5.2
	GET	/api/pronosticos/mis-pronosticos/filtrar	Autenticado	5.2
	GET	/api/pronosticos/recientes-acertados	Autenticado	Dash
	GET	/api/pronosticos/partido/{id}	Autenticado	5.3
GRUPOS	POST	/api/grupos	Autenticado	8.1
	POST	/api/grupos/unirse	Autenticado	8.3
	GET	/api/grupos/mis-grupos	Autenticado	-
	GET	/api/grupos/{id}	Autenticado	-
RANKING	GET	/api/ranking/global	Autenticado	7.1
	GET	/api/ranking/pozo	Autenticado	Dash
	GET	/api/ranking/grupo/{id}	Autenticado	8.4

15. Reglas de Negocio Implementadas

RF1 — Bloqueo de Pronósticos (30 minutos)

No se puede pronosticar si faltan menos de 30 minutos para el inicio. Calculado con `Instant.now().isAfter(inicioUtc.minusSeconds(1800))` en `Partido.estaBloqueado()`.

RF2 — Equipos

RF2.1: todos los autenticados pueden ver equipos. RF2.2: solo ADMIN puede crear/modificar. RF2.3: no se puede eliminar un equipo con partidos asociados (verificado con `@Query tienePartidosAsociados`).

RF3 — Fechas

RF3.1: CRUD básico. RF3.2: solo se modifica o elimina si está PROGRAMADA y sin partidos. RF3.3: estado calculado automáticamente con `recalcularEstado()` cada vez que un partido cambia de estado.

RF4 — Partidos

RF4.1: local y visitante deben ser distintos. RF4.2: solo se modifica si está POR_JUGARSE. RF4.3: pasa a EN_JUEGO por ADMIN o por el scheduler automático. RF4.4: listado ordenado cronológicamente. RF4.5: solo se elimina si está POR_JUGARSE y sin pronósticos.

RF5 — Pronósticos

RF5.1: UPSERT — si ya existe el pronóstico del usuario para ese partido, lo modifica; si no, lo crea. RF5.2: el usuario puede ver sus pronósticos filtrados por estado. RF5.3: pronósticos de terceros solo visibles después del bloqueo de 30 minutos.

RF6 — Resultados y Puntuación

RF6.1: solo se carga resultado si el partido está EN_JUEGO. RF6.2: tendencia calculada automáticamente (LOCAL / EMPATE / VISITANTE). RF6.3 motor: 3 puntos por resultado exacto, 1 punto por tendencia correcta, 0 puntos si no acierta ninguno.

RF7 — Ranking Global

RF7.1: ordenado por puntos totales descendente. RF7.2 desempates: 1) más plenos (exactos con 3 puntos), 2) primer pronóstico más antiguo (el que empezó antes en el tiempo).

RF8 — Grupos Privados

RF8.1: cualquier autenticado puede crear un grupo; el creador se agrega automáticamente como primer miembro. RF8.2: código de 8 chars único generado automáticamente (hasta 10 intentos). RF8.3: unión mediante código de invitación, sin duplicados. RF8.4: ranking exclusivo por grupo.

16. Motor de Puntuación — Diagrama de Flujo (RF6.3)

Se ejecuta en dos lugares: PartidoServiceImpl.registrarResultado() cuando el ADMIN carga el resultado, y FootballSyncScheduler.enJuegoJob() cuando el scheduler detecta un resultado en la API externa.

Ejemplo con resultado real 2-1 (LOCAL gana)

#	Paso	Detalle
1	Obtener resultado	golesLocal = 2, golesVisitante = 1 (del ADMIN o de la API).
2	Actualizar partido	setGolesLocal(2), setGolesVisitante(1), setTendencia(LOCAL), setEstado(FINALIZADO).
3	Buscar pronósticos	pronosticoRepository.findByPartidoid(id) — obtiene TODOS los pronósticos del partido.
4A	Exacto: pred 2-1	2==2 y 1==1 → TRUE → puntos = 3 (PLENO).
4B	Tendencia: pred 3-0	3==2 → FALSE → tendencia 3>0 = LOCAL == LOCAL → puntos = 1.
4C	Sin acierto: pred 0-2	0==2 → FALSE → tendencia 0<2 = VISITANTE != LOCAL → puntos = 0.
5	Persistir	pronosticoRepository.saveAll(pronosticos) — una sola transacción.
6	Recalcular fecha	fecha.recalcularEstado() — verifica si todos los partidos están FINALIZADOS.

17. Flujo de Autenticación JWT

Registro de nuevo usuario

4. Cliente envía POST /api/auth/register con {username, email, password}.
5. AuthServiceImpl valida unicidad de username y email.
6. Crea Usuario con password hasheada (BCrypt factor de costo 10).
7. Genera JWT con generarToken(username, 'USER') — expira en 24hs.
8. Devuelve {token, username, rol}. El frontend lo guarda y lo usa en los requests.

Login

9. Cliente envía POST /api/auth/login con {username, password}.
10. Busca usuario por username. Si no existe: 'Credenciales inválidas' (mensaje genérico).
11. passwordEncoder.matches(password, passwordHash). Si falla: mismo mensaje genérico.
12. Genera JWT y devuelve {token, username, rol}.

Request autenticado

13. Cliente envía request con header: Authorization: Bearer eyJhbGci...
14. JwtFilter.doFilterInternal() lee el header y extrae el token (quita 'Bearer ').
15. jwtUtil.esValido(token) → parsea y verifica la firma HMAC con la clave secreta.
16. Extrae username y rol. Crea UsernamePasswordAuthenticationToken con ROLE_USER o ROLE_ADMIN.
17. SecurityContextHolder registra la autenticación → Spring Security autoriza el acceso al endpoint.
18. El controlador llama a usuarioContextService.getUsuarioAutenticado() para obtener el usuario autenticado sin depender del body del request.

18. Cómo Ejecutar el Proyecto

Prerrequisitos

- Java 17+ instalado.
- Maven 3.9+ instalado.
- PostgreSQL corriendo (puerto 5433 por defecto).
- Base de datos creada: CREATE DATABASE prode_db;

Comandos

Windows (CMD)

```
set DB_PASSWORD=postgres
mvn clean spring-boot:run
```

Linux / Mac

```
DB_PASSWORD=postgres mvn clean spring-boot:run
```

Con perfil de desarrollo

```
mvn clean spring-boot:run -Dspring.profiles.active=dev
```

La aplicación arranca en: <http://localhost:8080>

Variables de entorno disponibles

Variable	Valor por defecto / descripción
DB_HOST	localhost
DB_PORT	5433
DB_NAME	prode_db
DB_USER	postgres
DB_PASSWORD	postgres
JWT_SECRET	Clave de mínimo 256 bits. Cambiar en producción.
FOOTBALL_API_TOKEN	Token de api-sports.io / football-data.org.
CORS_ORIGINS	* (desarrollo) o dominio específico en producción.

19. Repositorio de Código

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/Facualbri/TPI-PROG-4.git>

20. Diagrama de Base de Datos

El esquema relacional de ProdeUTN contiene 8 tablas principales generadas automáticamente por Hibernate (ddl-auto=update). A continuación se describe el modelo entidad-relación y sus relaciones clave.

Tablas y relaciones

Tabla	PK / PK compuesta	Claves Foráneas (FK) y restricciones destacadas
usuarios	id UUID	username UNIQUE, email UNIQUE. Referenciada por pronosticos, grupos_privados, grupo_miembros.
equipos	id UUID	nombre UNIQUE. Referenciada dos veces en partidos (equipoLocal_id, equipoVisitante_id).
fechas	id UUID	nombre UNIQUE. Referenciada por partidos (fecha_id FK).
partidos	id UUID	FK fecha_id → fechas. FK equipoLocal_id y equipoVisitante_id → equipos. api_external_id UNIQUE. Referenciada por pronosticos.
pronosticos	id UUID	FK usuario_id → usuarios. FK partido_id → partidos. UNIQUE(usuario_id, partido_id) garantiza un pronóstico por usuario por partido.
grupos_privados	id UUID	FK creador_id → usuarios. codigo_invitacion UNIQUE (8 chars). Referenciada por grupo_miembros.
grupo_miembros	PK compuesta: (grupo_id, usuario_id)	FK grupo_id → grupos_privados. FK usuario_id → usuarios. Tabla de relación M:N.
api_sync_log	id UUID	Tabla de auditoría independiente. Sin FK a otras tablas. Almacena historial de sincronizaciones con la API externa.

El diagrama ER completo en formato imagen (.png) y como archivo de diseño (drawio / dbdiagram.io) se encuentra en la carpeta /docs/db del repositorio.

21. Colección de Postman

Se incluye una colección Postman con los 31 endpoints de la API organizados por módulo. El archivo se encuentra en el repositorio en la ruta:

`/docs/postman/ProdeUTN.postman_collection.json`

Estructura de la colección

Carpeta	Endpoints	Variables de entorno necesarias
AUTH	2 (register, login)	{{base_url}}
EQUIPOS	5	{{base_url}}, {{token_admin}}
FECHAS	5	{{base_url}}, {{token_admin}}
PARTIDOS	7	{{base_url}}, {{token_admin}}, {{partido_id}}
PRONÓSTICOS	5	{{base_url}}, {{token_user}}
GRUPOS	4	{{base_url}}, {{token_user}}
RANKING	3	{{base_url}}, {{token_user}}

22. Informe del Frontend

Estructura general del frontend

El frontend se organiza en dos grandes carpetas bajo `assets/`: una subcarpeta `css/` y una subcarpeta `js/`. El punto de entrada es `static/index.html`, que carga todos los assets y ejecuta `App.init()` al cargar el DOM. Los scripts se cargan en orden: `helpers` → `state` → `auth` → `api` → `router` → `components` → `pages`.

5.1 Sistema de diseño CSS

El sistema de diseño se define mediante variables CSS en `tokens.css`. El tema oscuro (por defecto) usa `#0f172a` como fondo base, verde `#22c55e` como color primario y azul `#3b82f6` como acento. Las tipografías utilizadas son Inter (sans-serif) y JetBrains Mono (mono). El tema claro se activa con el atributo `[data-theme="light"]` y sobrescribe colores de fondo y texto.

El archivo `components.css` (649 líneas) define botones, cards, formularios, badges, spinners, tabs, avatares, skeletons y animaciones. Los botones `(.btn)` incluyen variantes primaria (verde), acento (azul), secundaria (ghost), danger y warning. Las cards tienen efectos hover con cambio de borde y sombra. El archivo `layout.css` (500 líneas) gestiona el sidebar (260px fijo), topbar mobile, match cards y ranking table. El archivo `pages.css` (1725 líneas) contiene estilos específicos por página, incluyendo el hero 3D de la landing con glassmorphism y scroll reveal.

5.2 Arquitectura JavaScript

La capa JavaScript sigue una arquitectura modular SPA (Single Page Application) con enrutamiento por hash. Los módulos principales son:

api.js: cliente HTTP genérico con JWT. Maneja 401 limpiando sesión, y expone métodos `get/post/put/patch/del`.

auth.js: manejo de sesión con `localStorage` (token + usuario), login, registro, logout y verificación de rol ADMIN.

state.js: estado global reactivo con patrón listener. Almacena usuario, fechas, equipos y tema.

router.js: SPA router por hash con 12 rutas registradas, guardas de autenticación y soporte para cleanup de páginas.

helpers.js: utilidades de formato de fechas, sanitización HTML, debounce, skeletons y manejo de errores.

Los componentes compartidos incluyen: `navbar.js` (sidebar + topbar con 8+ items, overlay mobile), `toast.js` (notificaciones con 4 tipos y auto-ocultación), `skeleton.js` (placeholders de carga) y `chart.js` (wrapper de Chart.js con gestión de destrucción para evitar memory leaks).

5.3 Páginas — Funcionamiento

El frontend cuenta con 12 páginas. A continuación se describen las principales:

LandingPage (516 líneas): página pública con hero 3D, equipos, partidos próximos y ranking. Usa `IntersectionObserver` para scroll reveal y contadores animados. Consume `GET /equipos, /partidos/proximos y /ranking/global`.

LoginPage (184 líneas): login y registro con tabs, validación de campos, toggle de contraseña y soporte de “Recordarme”.

DashboardPage (170 líneas): panel del usuario con 4 stat cards (pronósticos, aciertos, pozo, posición) y grid de últimos aciertos + ranking.

PartidosPage (257 líneas): listado con filtros por fecha, estado y equipo (debounce 300ms), paginación de 10 por página y envío de pronósticos vía POST /pronosticos.

AdminPage (~850 líneas): panel exclusivo para ADMIN con CRUD completo de equipos, fechas y partidos, importación desde API externa y botón “Actualizar estados” (POST /partidos/transicionar).

Otras páginas: MisPronosticos (tabs por estado), Historial (paginación + stats), Ranking (global + por grupos), Estadísticas (Chart.js doughnut + bar), Perfil (cambio de email y contraseña), Configuración (toggle tema) y Grupos (crear/unirse/salir).

5.4 Resumen de cambios realizados en el frontend

Los principales cambios respecto a versiones anteriores son los siguientes:

Archivo	Cambio principal
pages.css	Glassmorphism en todas las cards, hero más grande, escudos con glow, footer 2 columnas, secciones con gradient, .admin-section
components.css	Agregado .btn-warning, animaciones decorativas adicionales
landing.js	Hero con partidos reales (antes hardcoded), footer sin columna Contacto, equipos sin stagger delay
perfil.js	Promise.all → Promise.allSettled, guard de email sin cambios, refresco post-guardado
estadisticas.js	Promise.all → Promise.allSettled con valores por defecto
ranking.js	Promise.all → Promise.allSettled, caché _misGrupos
dashboard.js	Promise.all → Promise.allSettled con valores por defecto
admin.js	Botón “Actualizar estados”, .catch(() => []) en todas las API calls, _transicionarEstados()
login.js	“Olvidé contraseña” muestra mensaje “no disponible”
grupos.js	Escapado de backslash en onclick
partidos.js	Render condicional de inputs de predicción (solo POR_JUGARSE)
index.html	Referencias actualizadas a assets/css/ y assets/js/

23. Conclusión

ProdeUTN es una plataforma web de pronósticos deportivos desarrollada utilizando Java 17, Spring Boot, PostgreSQL y JWT. El proyecto implementa una arquitectura por capas que facilita el mantenimiento, la escalabilidad y la seguridad de la aplicación. Se desarrollaron funcionalidades de autenticación, gestión de equipos, fechas, partidos, pronósticos, rankings y grupos privados, además de integración automática con una API externa de fútbol. Como trabajo futuro, podrían incorporarse notificaciones en tiempo real, análisis estadísticos avanzados y despliegue en infraestructura cloud.